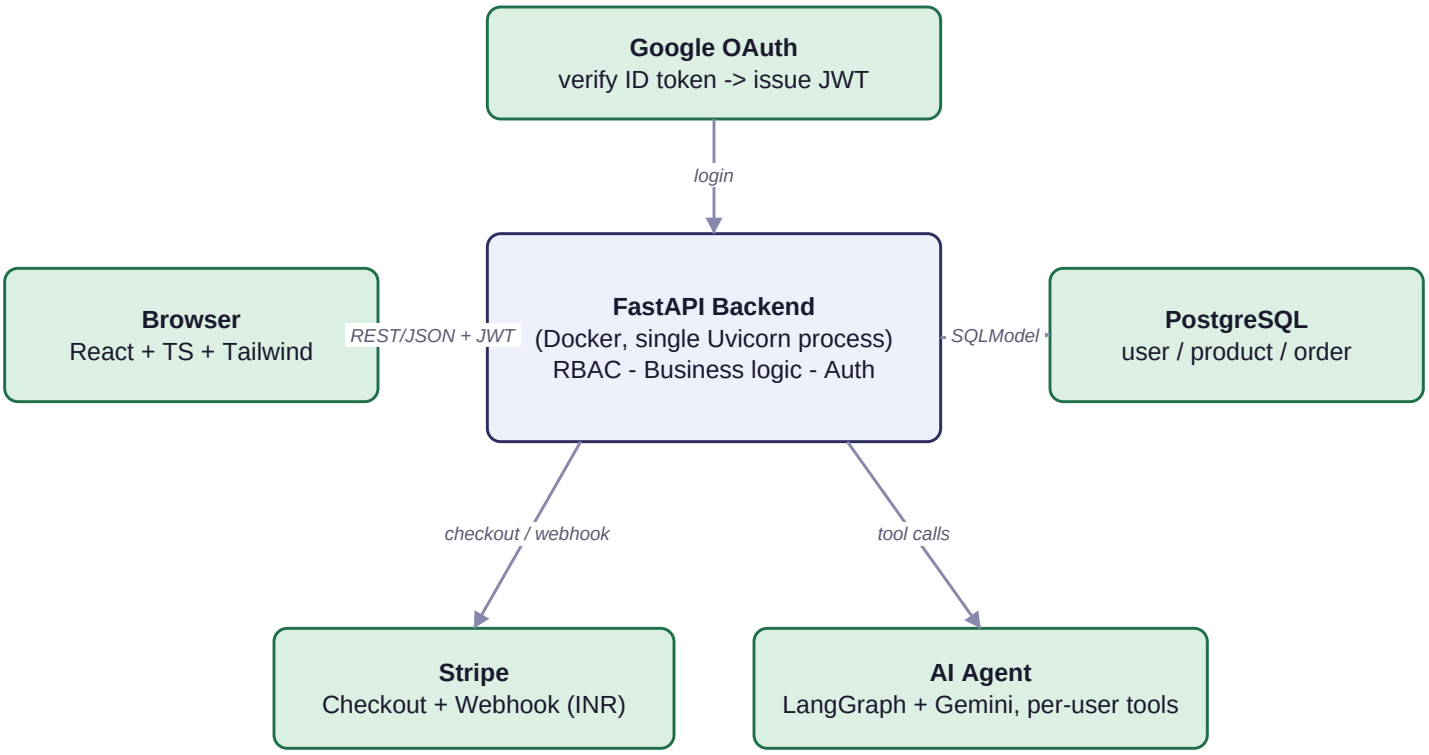
System Design - Mini AI E-Commerce Application

AI Full Stack Developer Technical Assignment - Assignment 2

Live app: mini-ai-ecommerce.vercel.app | Live API: mini-ai-ecommerce-backend.onrender.com | Repo: github.com/diiisha24/mini-ai-ecommerce

Architecture



AWS Deployment Approach

(not deployed for this submission - built to run locally, see README)

- Frontend: static Vite build on S3 + CloudFront (or Amplify Hosting).
- Backend: Docker image on ECS Fargate behind an Application Load Balancer; secrets from AWS Secrets Manager, never baked into the image.
- Database: Amazon RDS for PostgreSQL (single-AZ for this scale).
- Stripe webhook: ALB target must be public HTTPS (ACM cert) since Stripe calls it directly.
- CI/CD: GitHub Actions builds the image, pushes to ECR, deploys to ECS on merge to main.

Scaling Considerations

- Current ceiling, stated concretely: single Uvicorn process + SQLAlchemy's default pool = ~15 concurrent DB-bound requests before queuing - that's the real trigger point, not a guess.
- Backend is stateless (JWT auth, no server sessions) - scales horizontally by adding ECS tasks behind the ALB.
- DB: read replica for product listing; Redis cache for the catalog and for AI tool outputs (e.g. list_products) once read volume grows well past write volume.
- AI agent (most latency/cost-sensitive part): move chat off the request path onto SQS + an async worker under bursty load; add per-user rate limiting to cap LLM spend; track token cost per call.
- Stripe webhooks already idempotent under a row lock (SELECT ... FOR UPDATE) so Stripe's automatic retries can't double-process an order.
- Observability: structured logging + request tracing (OpenTelemetry) once traffic outgrows eyeballing logs.

Full detail: docs/system-design.md, docs/db-schema.md, docs/api.md in the repository.